

S3 Lab Work

An evidence-traceable review system for Indian cars and two-wheelers. Every number on the site opens the sentences it was computed from.

SUBMISSION

S3 Lab work

DUE

Friday 2 October 2026

TEAM

Aditya Nariyapara, Devika Jonjale, Saachi Shinde

MEASURED

18 September 2026, on `main`

LIVE APPLICATION

revix-reviews.vercel.app

REPOSITORY

github.com/saaachis/revix

What this report covers

Five sections, in the order of the assessment breakdown. Section three gives each measuring tool a sheet of its own.

1	Codebase	Shape, the pipeline, and what it serves	3–6
2	Frameworks, with justification	Choices, what we rejected, where it runs	7–10
3	Code quality, measured	Six tools, one sheet each, and the reports	11–19
4	Non-functional requirements	All nine, audited	20–22
5	Run it live	Reproduce every figure	23

ASSESSMENT BREAKDOWN

CODEBASE	10
17,600 lines	
Three Python packages and a Next.js app, on real data	
FRAMEWORKS	10
9 decision records	
Every choice written down, including what we rejected	
CODE QUALITY	10
Grade A, all 43 files	
Measured with Radon, enforced by Xenon in CI	
NFRS ACHIEVED	10
9 of 9 audited	
Six met by one decision, five gaps found and closed	

HOW TO READ THE NUMBERS

Every figure in this document was produced by running the tool named beside it. Page 13 gives the commands, so any number here can be checked in under a minute rather than taken on trust.

Codebase

A uv workspace: three Python packages and one Next.js application, running unattended on reviews collected from three sources.

SHAPE

revix/

└─ packages/revix_core/

└─ pipeline/

└─ apps/api/

└─ apps/web/

└─ tests/

└─ db/migrations/

└─ docs/

└─ .github/workflows/

models, settings, session

ingest → fuse → publish

serving layer + operations

Next.js 16 frontend

16 modules, 308 tests

Alembic, reversible

ADRs, audits, this report

CI, nightly, probe, keep-warm

1,312

6,716

1,516

5,069

2,990

58 Python files · 24 TypeScript files · ~17,600 lines

RUNNING ON REAL DATA, NOT FIXTURES

Evidence units collected	20,263
Variants in the catalogue	143
Variants with a published verdict	134
Held back below the evidence floor	9
Models · manufacturers	42 · 16
Last full pipeline run	53m 28s

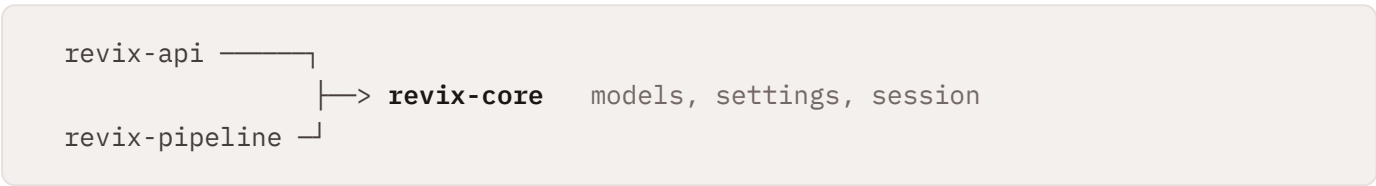
Owner reviews from CarWale and BikeWale, CarDekho and BikeDekho, plus comments on YouTube review videos. Three sources because three permit us: Team-BHP returns 403 to our agent and two other sites disallow us outright in `robots.txt`, and we chose to respect that rather than spoof a browser.

Structure that a build can enforce

THE ONE RULE THAT HOLDS IT TOGETHER

Dependencies run in **one direction only**. The API and the pipeline both depend on `revix_core`; core depends on neither, so the read path and the write path cannot tangle.

Enforced by import linting in CI, not by us remembering. In a single package this would be a naming convention, and naming conventions lose. This is also why it is three packages rather than one: the API physically cannot import the pipeline.



THE DATABASE MIRRORS THE SAME IDEA

Four Postgres schemas separated by lifecycle rather than by subject, so it is always obvious which layer a table belongs to and what is allowed to write to it.

SCHEMA	HOLDS	WRITTEN BY
raw	Fetches payloads, content-hashed	Ingest
core	Catalogue, sources, evidence units	Ingest and resolution
analysis	Aspect scores, evaluation runs	Enrichment and fusion
serving	Finished verdicts, claims, citations	Fusion; read by the API

SCHEMA CHANGES ARE CHECKED THREE WAYS

- The migrations **apply** from empty.
- They **match the models**, so editing a model without a migration is a red build.
- They **reverse cleanly**, which is the one most projects never test.

The pipeline, stage by stage

Five stages, each a separate command, each runnable and debuggable on its own.
The nightly simply runs them in order.

INGEST Fetch from each source, politely. Store the response, deduplicate by hash	EXTRACT Split into sentences, find the topic and the polarity of each	RESOLVE Decide which vehicle a listing is about, or refuse and queue it	FUSE Weight, score, bootstrap the interval, apply the evidence floor	PUBLISH Write verdicts, claims and citations into the serving schema
---	---	---	--	--

WHAT EACH STAGE GUARANTEES

STAGE	THE PROPERTY IT HOLDS	WHY IT MATTERS
Ingest	Idempotent, and isolated per source	Re-running costs a hash lookup, not a duplicate. One dead source never stops the others
Extract	Deterministic, no model call	The same sentence always yields the same topic, so a verdict can be reproduced
Resolve	Refuses below its confidence floor	An unsure match becomes a queued decision for a person, never a guess in the data
Fuse	Writes citations before prose exists	The score is computed <i>from</i> the cited rows, so a citation cannot be wrong
Publish	One row per variant per weighting	Switching weighting on the site is a lookup, never a recomputation

POLITENESS IS BUILT IN, NOT BOLTED ON

Every outbound request passes a rate limiter, a `robots.txt` check and a circuit breaker that stops asking after repeated refusals. Three sources are read because three permit it: Team-BHP returns 403 to our agent and two others disallow us outright. **We respect that rather than spoof a browser.**

What it serves, and what tests it

API SURFACE: 15 ENDPOINTS, 7 PUBLIC AND 8 OPERATIONAL

ENDPOINT	RETURNS
GET /variants	The catalogue, filtered and paginated, capped at 200
GET /variants/{id}/verdict	The endpoint. One indexed read, no computation
GET /claims/{id}/evidence	The sentences behind one claim, ranked by contribution
GET /fusion-configs	The three weightings the switch offers
GET /metrics	Every measurement recorded, including unflattering ones
GET /sources/health	Where each source stands, so a gap is visible
GET /health	503 when the database is unreachable, not a stack trace
/admin/*	Eight operational routes, authenticated, never cached

WEB SURFACE: 11 ROUTES, ALL SERVER-RENDERED

Landing and search	/ • /search	Catalogue and comparison	/browse • /compare
The verdict and its evidence	/v/[id] • /evidence/[id]	How and how well	/method • /accuracy
Where it comes from	/sources • /status	Operations, gated	/admin

TEST SUITE: 19 MODULES, 312 TESTS

WHAT IS COVERED	MODULES	TESTS
Pipeline, fusion and round trips	3	60
Connectors and live sources	4	87
API, CORS, health and admin auth	4	36
Non-functional guarantees, quota, retention	4	39
Schema, reference data, evidence model	4	90

266 run without a database; CI runs the full suite against a real Postgres with pgvector, and also checks the migrations apply, match the models, and reverse cleanly.

Frameworks, with justification

The artefact for this mark is the decision-record folder: nine ADRs, each stating the decision, the alternatives considered, and the consequence we accepted.

CHOICE	WHY THIS ONE	WHAT WE DID NOT DO
FastAPI	Contract-first. The TypeScript client is generated from the OpenAPI schema, so the two <i>cannot</i> drift	Flask, and hand-syncing the client
Pydantic v2	Validation at the boundary, so a malformed request never reaches a query	Manual checks scattered through handlers
SQLAlchemy 2.0	Typed ORM with bound parameters everywhere, so injection has no surface	Raw SQL: quicker to write, impossible to refactor safely
PostgreSQL + pgvector + pg_trgm	One datastore covers relational, vector <i>and</i> fuzzy text. Resolution needs trigrams; deduplication needs vectors	A separate vector database, a second service to keep consistent
Next.js 16 App Router	Server rendering, so a verdict is a real URL that can be shared and indexed	A client-only single-page app
Tailwind v4	CSS-first theme, so the palette lives in one place. It was reworked once for contrast, and one place is why that was cheap	Ad-hoc stylesheets per component
Typer	The pipeline is a CLI, so every stage is runnable and debuggable on its own	A monolithic script

THE THREAD RUNNING THROUGH ALL OF THEM

Each of these was chosen so that a mistake becomes a failed build rather than a defect a reader finds. The generated client, the typed ORM, the migration checks and the contract-first schema are all the same move: **make the wrong thing impossible to merge** rather than rely on noticing it.

The two choices worth arguing about

ADR 0002 · ORCHESTRATION

Scheduled GitHub Actions instead of Airflow or Prefect. An orchestrator is a server to run, secure and pay for. Our pipeline is one nightly batch: no fan-out, no backfill, no dynamic DAG. Actions already holds our secrets, already runs on a schedule, and already gives us logs and alerting.

Choosing Airflow here would have been choosing a tool because it looks professional rather than because the problem needs it. If we ever needed backfill or fan-out, that ADR is where the decision gets revisited.

ADR 0004 · MACHINE LEARNING, AND THE HONEST RESULT

We built a distant-supervision aspect classifier: lexicon labels train a TF-IDF into a one-vs-rest logistic regression.

It lost to the lexicon it learned from by 0.29 macro F1, so it ships disabled and the number is published on our public metrics page rather than quietly dropped.

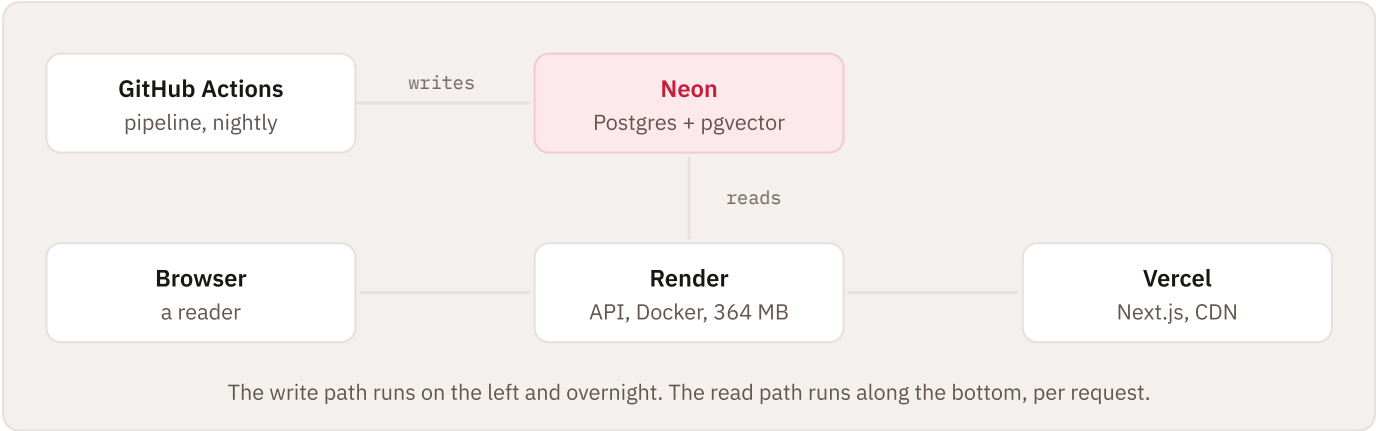
The alternative was shipping a model because we had built one, and the whole argument of this project is that a number should be earned.

THE NINE DECISION RECORDS

0001	Record architecture decisions
0002	Scheduled CI instead of Prefect
0003	A uv workspace with three packages
0004	A lexicon baseline before a classifier
0005	The frontend stack, and why the switch is instant
0006	Official APIs only, and what we checked before deciding
0007	How the fusion experiment avoids fooling us
0009	Two sources, not three
0010	Basic auth for the operations surface

Where it runs

Four platforms, each doing the one thing it is best at, and none of them required for a page to render except the database.



PLATFORM	RUNS	CHOSEN BECAUSE
GitHub Actions	The nightly pipeline, and CI	Already holds the secrets and the schedule. No orchestrator to run or pay for
Neon	Postgres 16, pgvector, pg_trgm	Serverless Postgres with the extensions we need, and a free tier
Render	The API, as a Docker image	Container hosting with a health-check contract we already satisfy
Vercel	The Next.js frontend	Static assets on a CDN, so the site stays up independently of the API

The development toolchain

Chosen on one principle: a mistake should become a failed build rather than a defect a reader finds.

TOOL	JOB	ENFORCED WHERE
uv	Workspace and lockfile across three packages	CI installs from <code>uv.lock</code> , frozen
Ruff	Lint and format, replacing black, isort and flake8	Fails the build
mypy	Static types, strict, all three packages	Fails the build
Radon	Maintainability and complexity scores	Printed every run
Xenon	The same scores, as a ceiling	Fails the build
PyTest + coverage	312 tests, coverage measured	Fails the build
Alembic	Migrations, applied and reversed	Fails the build
openapi-typescript	Generates the frontend client from the schema	Fails if the committed client drifts
Playwright	Browser smoke test, 10 checks	Fails the build
axe-core	WCAG 2.1 AA across all 11 routes	Fails on a violation

THE CHECK WORTH DEFENDING HARDEST

CI regenerates the TypeScript client from the live OpenAPI schema and **fails if the committed types differ**. A response shape that changes without the frontend changing is a red build, not something a user discovers.

It is structural rather than procedural: it does not depend on anybody remembering.

SIX CI JOBS, ON EVERY PULL REQUEST

Repository hygiene	secrets, payloads, links	API and pipeline	lint, types, gate, tests
Web	lint, types, build, a11y	API image	Docker builds
Migrations	apply, match, reverse	Pipeline end to end	runs on fixtures

Code quality, measured

Put through a review toolkit rather than described in prose. **Radon** produces the score, **Xenon** enforces it, **Ruff** and **mypy** cover lint and types, **PyTest** runs the suite.

A

• all 43 files

MAINTAINABILITY INDEX

Radon. Every file grade A, no exceptions

3.90

AVERAGE CYCLOMATIC COMPLEXITY

Grade A, across 344 blocks

0

BLOCKS GRADED E OR F

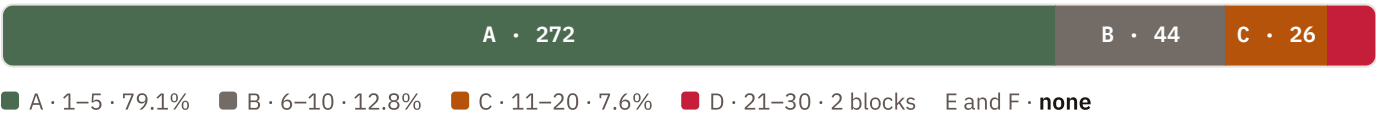
Was one before this review. See page 8

308 • 64%

TESTS • COVERAGE

Ruff, format and mypy strict all clean

COMPLEXITY DISTRIBUTION, 344 BLOCKS



WHAT EACH TOOL MEASURES

TOOL	MEASURES	RESULT
Radon	Maintainability index, cyclomatic complexity, raw counts, Halstead	A throughout
Xenon	The same measurement, wired to an exit code	gate passes
Ruff	Lint and formatting, one tool instead of three	clean
mypy	Static types, strict, all three packages	clean
PyTest	308 tests, coverage measured in CI	64%

Maintainability index

A · all 43 files Every source file is maintainable, with no exceptions.

The weakest file still scores 24.17, four points clear of the grade boundary.

```
uv run radon mi packages/revix_core/src pipeline/src apps/api/src -s
```

WHAT THE NUMBER IS

A single score from 0 to 100 per file, from Halstead volume (how much there is to read), cyclomatic complexity (how many paths through it) and lines of code. It answers **how hard this file will be to change in six months**.

A 20 to 100 maintainable	B 10 to 19 moderate	C 0 to 9 difficult
---------------------------------------	----------------------------------	---------------------------------

WHERE OUR FILES SIT

GRADE	FILES	SHARE
A · maintainable	43	100%
B · moderate	0	0%
C · difficult	0	0%

THE FOUR LOWEST, AND WHY

FILE	SCORE	WHAT IT DOES
<code>cli.py</code>	A (24.17)	Thirty-odd small commands in one module. Long rather than complicated
<code>enrichment/fuse.py</code>	A (34.77)	The scoring engine. The densest logic in the project, and the most tested
<code>evaluation/fusion_experiment.py</code>	A (39.95)	The evaluation harness, refactored during this review
<code>catalogue_discovery.py</code>	A (43.89)	Reads two sites' markup, so it carries the shape of somebody else's HTML

The three lowest scores are the three files that do the most genuine work. That is the metric behaving as intended, and a reason to trust it here rather than be alarmed by it.

Cyclomatic complexity

3.90 · grade **A** **Average across 345 blocks, in the simplest band.**

Nearly four fifths of the codebase is grade A, and nothing scores E or F.

```
uv run radon cc packages/revix_core/src pipeline/src apps/api/src -a -s -nC
```

WHAT THE NUMBER IS

The count of independent paths through a block of code, which is also **the minimum number of test cases needed to cover it**. A function scoring 24 has twenty-four ways through, so it needs at least twenty-four tests to exercise fully.

DISTRIBUTION ACROSS 345 BLOCKS



GRADE	BLOCKS	SHARE	READS AS
A	272	78.8%	Simple, low risk
B	45	13.0%	Well structured, stable
C	26	7.5%	Slightly complex, worth watching
D	2	0.6%	Complex, justified on the next page
E, F	0	0%	Would need fixing. There are none

91.8% of the codebase is A or B. The 26 blocks at C are mostly source parsers, which branch because real-world markup does.

The measurement changed the code

THE ONLY E GRADE IN THE CODEBASE

One function scored **E (33)**, forty percent worse than the next worst block. Reading it, the cause was plain: `_evaluate` drew samples from the corpus *and* turned those observations into summary statistics. Neither half is complicated. Doing both in one place was.

Split along that seam into five functions — `_draw`, `_score_one_draw`, `_rank_correlation`, `_calibration` and `_summarise` — with a small value object carrying observations between the two halves. The thirteen tests covering it passed unchanged throughout, which is the only reason a refactor there was safe to make.

BEFORE AND AFTER

	BEFORE	AFTER
Worst block	E (33)	D (24)
Blocks at E or worse	1	0
Average complexity	3.944	3.895

METHOD, NOT LUCK

Measure, find the worst thing, understand *why* it is the worst thing, fix the one that is safe to fix, and document the ones that are not.

THE TWO BLOCKS STILL AT D, DELIBERATELY

BLOCK	WHY THE COMPLEXITY IS THE DOMAIN'S, NOT A STYLE ACCIDENT
<code>fuse_variant</code> D (24)	The scoring engine: weighting, bootstrap interval, evidence floor, suppression, citations. Splitting it into five functions that only ever run in sequence would move the complexity into the call graph rather than remove it. It is the most heavily tested function in the project.
<code>run_connector</code> D (21)	The ingest loop, holding three failure modes open at once so that one source failing never stops the others. That branching is the resilience requirement.

Both sit on the critical write path, and a refactor that changes a verdict is worse than a D grade. A threshold should be set where the code is and then defended, not chased.

Enforcing the score, and the style

XENON: THE SAME MEASUREMENT, WIRED TO AN EXIT CODE

pass • gate holds Radon reports; Xenon refuses.

A pull request that makes any grade worse does not merge.

```
uv run xenon --max-absolute D --max-modules B --max-average A \
  packages/revix_core/src pipeline/src apps/api/src
```

CEILING ON	SET TO	WHY THERE
Any single block	D	Where the code is; nothing may get worse than the two justified functions
Any single module	B	A module may contain a hard function, not become a hard module
The whole codebase	A	The average must stay in the simplest band

Thresholds are set **where the code actually is**, so they catch regression rather than flatter us. That is the difference between a measurement and a property.

RUFF: LINT AND FORMAT, ONE TOOL INSTEAD OF THREE

0 • findings Clean across 98 files, and the formatting is checked too.

Replaces black, isort and flake8, so style is never a review conversation.

```
uv run ruff check .          # 0 findings
uv run ruff format --check . # 98 files already formatted
```

Both run in CI and both fail the build. Formatting being enforced rather than agreed is what keeps diffs about behaviour instead of whitespace.

Static types, strict

0 • errors in 43 files **Strict mode, across all three Python packages.**
No implicit `Any`, no untyped function bodies, no unchecked calls.

```
uv run mypy packages/revix_core/src pipeline/src apps/api/src
uv run mypy ... --html-report build/reports/typing # the official report
```

TYPE PRECISION, FROM MYPY'S OWN REPORT

89.6% • precisely typed **10.44% imprecise across 10,189 lines.**
Imprecise means an expression mypy could only narrow to `Any`, usually at a library boundary rather than in our own logic.

WHERE THE IMPRECISION IS	IMPRECISE	WHY
<code>ml/aspect_model</code>	7.95%	scikit-learn returns loosely typed arrays
<code>ml/gold</code>	8.25%	Sampling over the same arrays
<code>reference</code>	8.97%	Seed data read from JSON
Whole codebase	10.44%	Concentrated at third-party edges

WHY STRICT IS WORTH THE FRICTION
A typed ORM, typed response models and a generated client only pay off if the types are actually checked. Strict mypy is what turns **"the API returns a different shape now"** into a failed build rather than a runtime surprise.

Tests, and how much they reach

310 • passing, 2 skipped **Run against a real Postgres with pgvector in CI.**

266 of them need no database at all, so they run in a second on any laptop.

```
uv run pytest --cov --cov-report=term
uv run pytest --cov --cov-report=html:build/reports/coverage  # the official report
```

65% • line and branch coverage **Measured in CI, where the database-backed tests also run.**

Deliberately not gated, and the next page explains why.

WHAT THE SUITE ACTUALLY COVERS

AREA	MODULES	TESTS
Schema, reference data, evidence model	4	90
Connectors and live sources	4	87
Pipeline, fusion and round trips	3	60
Non-functional guarantees, quota, retention	4	39
API, CORS, health, admin authentication	4	36

WHY COVERAGE IS NOT A GATE

A coverage threshold is trivially satisfied by tests that execute code without asserting anything about it. **We would rather the number stayed honest than stayed high**, and judge the suite by what it has caught instead.

The tool we left out, and what answers it instead

Five tools were used. The sixth was considered and rejected, which is worth stating rather than leaving as a silent omission.

MUTATION TESTING

Mutmut changes your code on purpose and asks whether any test notices, which measures something no other tool here does: not whether tests exist, but whether they would catch a bug.

It has no native Windows support, and all three of us develop on Windows. A quality tool only one environment can run is one that stops being run.

WHAT ANSWERS THE SAME QUESTION

Whether the tests would catch a bug, answered with evidence rather than a score: **five real defects the suite caught before they reached anybody.**

DEFECT	WHY IT MATTERED
Macro-F1 averaged over zero-support aspects	A perfect system scored 0.22. Our evaluation flattered us
A connector recorded our own search term as the source's answer	We would have been citing ourselves
An unordered <code>LIMIT</code> in fusion	Ingest and fusion disagreed about which variant they meant
Security headers added inside the rate limiter	A 429, the response most likely to reach a hostile client, went out bare
A migration emitting <code>NOT NULL</code> with no default	Would have failed against 129 production rows

The reports the tools generate

Two of the six tools produce a browsable report of their own. Both are generated from the same run as the figures in this document, and both can be opened in the session.

REPORT	FROM	WHAT IT SHOWS
Coverage build/reports/coverage/	coverage.py, via pytest-cov	Every file, every line, coloured by whether a test reached it. Click into a file to see exactly which branches were missed
Type precision build/reports/typing/	mypy --html-report	Per module, the share of expressions mypy could type precisely, with the imprecise ones highlighted in the source
Complexity data build/reports/complexity.json	Radon --json	Machine-readable rather than browsable. Radon also emits XML and Code Climate format for CI tooling

GENERATING THEM

```
uv run pytest --cov --cov-report=html:build/reports/coverage
uv run mypy packages/revix_core/src pipeline/src apps/api/src \
  --html-report build/reports/typing
uv run radon cc packages/revix_core/src pipeline/src apps/api/src -j \
  > build/reports/complexity.json
```

WHAT HAS NO REPORT, AND WHY THAT IS FINE

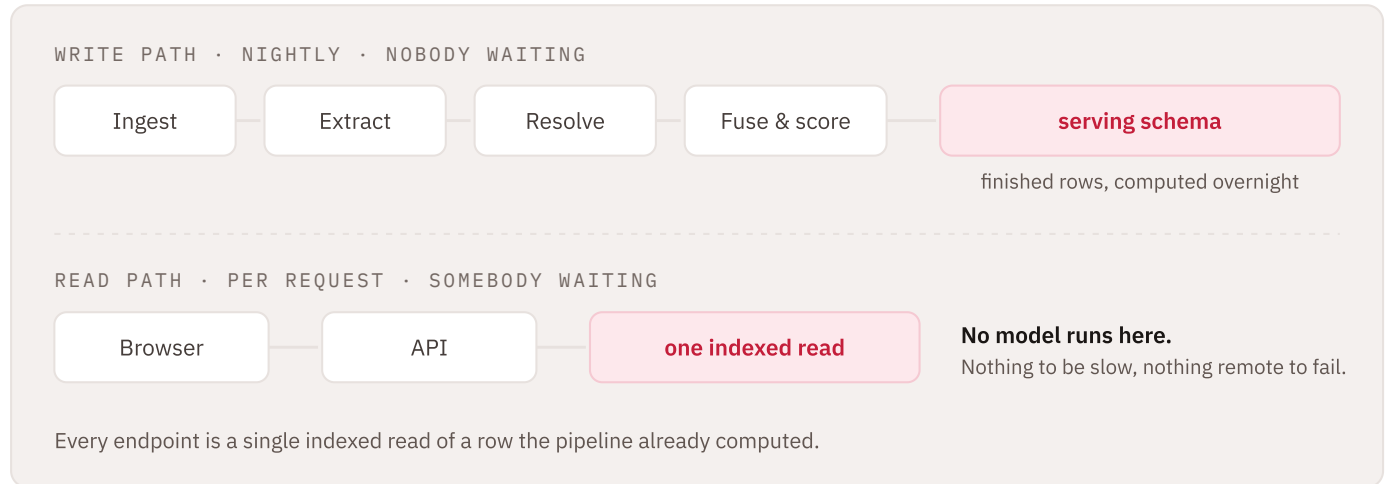
Ruff and **Xenon** report by exit code rather than by document, which is the right shape for a gate: the answer is pass or fail, and the build already carries it. Ruff can emit JSON or a statistics summary if a dashboard ever needs one.

Coverage report	65%, browsable	Type precision report	89.6% precise
Complexity export	345 blocks, JSON	Gates that only exit	Ruff, Xenon

Non-functional requirements

All nine categories audited against the actual code rather than against what the proposal claimed. **Six were already satisfied by a single architectural decision**, and that decision is worth explaining before the list.

THE WRITE PATH AND THE READ PATH ARE STRICTLY SEPARATED



22–35 ms

WARM RESPONSE, LOCAL

Against a 300 ms p95 target

96–247 ms

FROM INDIA, DEPLOYED

Inside target including the network

That one decision delivers four requirements at once. **Performance**, because nothing on the read path grows with the corpus, since the corpus is not on the read path. **Scalability**, because going from twenty thousand review units to a hundred thousand changes how long the night takes and changes nothing about a request. **Reliability**, because a page cannot fail when a third-party model is down. And **availability**, because a demo depends on nothing remote.

All nine, and where each is met

1 • Performance

met

22–35 ms warm, 96–247 ms from India against a 300 ms target. Gzip 9.3×, cache directives, and `X-Response-Time-ms` on every response so the claim is checkable from outside

2 • Scalability

met

Pagination capped at 200, bounded connection pool, the expensive work batched overnight where nobody is waiting

3 • Portability

met

Docker image 364 MB running as a non-root user, uv lockfile, configuration only through environment variables. Developed on Windows, built and tested on Linux

4 • Usability

audited

WCAG 2.1 AA clean on **all ten pages**, axe-core enforced in CI. The palette was reworked once because contrast failed, and the audit is what caught it

5 • Compatibility

met

The TypeScript client is generated from the live schema, so the frontend cannot drift from the API. Responsive layout, standards only

6 • Security

met

CSP and HSTS, CORS allow-list, rate limiting, no raw SQL, no secrets in a public repo, read-only public API and an authenticated operations surface

7 • Reliability

met

Sources degrade independently behind a circuit breaker, every database wait is bounded, and no traceback ever reaches a caller

8 • Maintainability

met

Strict types, 308 tests, nine decision records, a generated client, and configuration in one place

9 • Availability

stated honestly

A 503 health contract and a keep-warm schedule. One free-tier instance is *resilient to its dependencies failing*, not *highly available*

MEASURED, NOT ASSERTED

Accessibility 0 violations across ten pages · catalogue payload 55.3 kB down to 5.9 kB gzipped · API image 364 MB, non-root · thirteen tests in `test_nfr.py` assert the headers, the limiter, the error shape and the compression, because these are exactly the things that work the day they are added and quietly stop working six commits later.

The finding worth leading with

A REQUIREMENT WE HAD WRITTEN, TESTED, AND STILL NOT MET

Our `/health` endpoint is deliberately built to report an unreachable database as a **503** rather than a 500 with a stack trace. It was written that way on purpose. There is a test for it.

It could not actually do it. There was no connect timeout, so the driver sat on the operating system's TCP timeout while the hosting platform's health probe gave up first. A correct 503 became no answer at all — and a hang is worse than an error, because nothing downstream can react to it.

What led us there was our own test run hanging for four hundred seconds against a machine with no database. Every wait is now bounded. The lesson is the difference between *writing* a non-functional requirement and *verifying* one: we had the code, the intent and the test, and the requirement was still not met.

TWO LIMITS WE STATE RATHER THAN HIDE

LIMIT	WHY IT IS THE RIGHT CALL AT THIS SIZE
The rate limiter is per instance and in memory	It bounds what one instance absorbs rather than enforcing a global quota, which fits the actual risk: one scraper exhausting a connection pool, not a distributed attack. A global quota needs Redis, and adding a network dependency to the read path to protect the read path is a bad trade here.
A single free-tier instance is resilient, not highly available	There is no redundancy and no failover; buying that means two instances and a load balancer. Our mitigations target the failure that actually threatens a demo, a cold start or a sleeping database, rather than the one that does not.

Both are the first things that would need replacing if Revix grew. Stating a limit before being asked reads as engineering judgement; being caught at it does not.

Run it live

Every figure in this document is reproducible in under a minute. A number produced in front of you is worth more than a number in a document.

THE SCORE

```
# maintainability index, per file
uv run radon mi packages/revix_core/src pipeline/src apps/api/src -s

# cyclomatic complexity: the average, plus anything above B
uv run radon cc packages/revix_core/src pipeline/src apps/api/src -a -s -nC

# raw counts
uv run radon raw packages/revix_core/src pipeline/src apps/api/src -s
```

THE GATE CI RUNS

```
uv run xenon --max-absolute D --max-modules B --max-average A \
  packages/revix_core/src pipeline/src apps/api/src
```

THE REST OF THE QUALITY GATE

```
uv run ruff check . && uv run ruff format --check .
uv run mypy packages/revix_core/src pipeline/src apps/api/src
uv run pytest --cov --cov-report=term
cd apps/web && npm run all
```

LIVE SURFACES

[Live site](#)
[API health](#)
[Source health](#)
[OpenAPI](#)
[Repository](#)

SUPPORTING DOCUMENTS

[docs/s3-lab-work/03-code-review-report.md](#) — the full toolkit report

[docs/non-functional-requirements.md](#) — all nine categories in detail

[docs/adr/](#) — the nine architecture decision records

Revix · S3 Lab Work · measured 18 September 2026 on [main](#) . Figures grow with every pipeline run; the commands above reproduce all of them.